Bi-Level Harness Optimization and Textual Backpropagation in Autonomous 4-Loop Recursive Self-Improvement Engines

Yasemin Gavares & Alexander Gavares
Autonomous Agent Architecture Research Group
Local Service App Inc / Kronoz Technologies LLC
Technical Report 2026

Abstract

Autonomous Recursive Self-Improvement (RSI) in large language model (LLM) agents has historically suffered from reward hacking, oscillating regressions, catastrophic context collapse, non-differentiable search space stagnation, and exploration entropy collapse. We present a formal framework for Bi-Level Harness Optimization governed by an airtight 4-Layer Defense-in-Depth Execution Substrate. Instead of modifying foundational model weights during inner-loop search, an outer meta-optimization loop searches over a structured, capacity-bounded cognitive harness space $\mathcal{H} \in \mathcal{S}_{\mathcal{H}}$—comprising topological control flow, node prompts, runtime verification contracts, and a crystallized skill library—via Causal Program Slicing and Counterfactual Textual Backpropagation over Directed Acyclic Execution Graphs (DAGs). We incorporate physical resource proxies for algorithmic scalability and AST mutation elasticity directly into a 5-dimensional Pareto fitness vector, prove almost-sure monotonic convergence under a Two-Timescale Quality-Diversity MAP-Elites formulation, formalize adversarial curriculum synthesis as an asymmetric Stackelberg regret-maximization game utilizing constructive complexity separation and metamorphic invariants, and derive an information-theoretic lower bound on token complexity reduction achieved by domain-indexed System-2 to System-1 LoRA weight distillation via Reinforcement Learning with Verifiable Rewards (RLVR). Finally, we validate the architecture across a 43-generation multi-epoch evolution curriculum and four physical execution phases: (1) achieving 100% Vault pass rate convergence across all five benchmark domain archetypes in Generations 1–30 and a theoretical policy compression bound of $\rho_{\tau,\text{bound}} = 4.1498$; (2) distilling harvested trajectories into `Qwen2.5-Coder-1.5B-Instruct` LoRA weights via GRPO, attaining an 85.00% (17/20) zero-shot and 100.00% (20/20) contract-specified holdout Vault pass rate with a 2.8034× mean (4.91× peak) empirical token compression ratio; (3) executing self-referential AST surgery on the Meta-Harness orchestration DAG to achieve a 6.15× end-to-end latency speedup, eliminate 92.5% of redundant System-2 invocations, and intercept 100% of adversarial gate-bypass mutations; (4) closing the bi-level loop into Macro-Epoch 2 (Generations 31–35) to achieve a 100.0% (5/5) greedy zero-shot ($T = 0$) pass rate on unseen cross-domain hybrid specifications with a 0.0176 ms (576, 976×) amortized replay latency; and (5) extending the substrate across Generations 36–43 to validate capacity-bounded skill crystallization (5/5 primitives admitted, 100% second-order composition), oracle-free Hypothesis metamorphic verification (400/400 randomized trials passed, 100% reward-hack blindspot interception), and transactional multi-file repository evolution (`SWE-EVO`, 100% valid multi-module commit rate at $\bar{\Delta}_{\text{AST}} = 0.4845$ and 100% atomic rollback on cross-file regressions).

1 System Topology: The 4-Loop Architecture

Recent advances in automated agent design [5, 13] and programmatic prompt optimization [7, 14] demonstrate that agentic scaffolding heavily dictates downstream reasoning performance. However, unconstrained self-modification rapidly degrades into degenerate reward hacking [1]. Our architecture unrolls RSI across four distinct temporal timescales ($\Delta t_1 \ll \Delta t_2 \ll \Delta t_3 \ll \Delta t_4$) to isolate short-term execution from long-term architectural, library, and parametric evolution:

- **Loop 1 (Inner Execution DAG, $\Delta t_1 \sim 10^1$s):** A multi-node pipeline generates executable Python solutions (c = `target.py`) for a problem instance p by iterating against Tier 1 Public Tests, enforcing node-level pre/post-condition contracts, importing verified primitives from the crystallized skill library, and emitting structured execution traces (`trace.json`).
- **Loop 2 (Meta-Architect Outer Loop, $\Delta t_2 \sim 10^3$s):** Mutates `inner_harness.py` (control flow topology, node prompts, local verification policies) and extracts reusable, stateless helper abstractions into `stdlib_primitives.py`. It consumes `trace.json` and `sanitized_diagnostic.json`, isolates causal failure nodes via deterministic topological state bisection, logs structural blind spots to the SQLite Issue Registry, and applies surgical Abstract Syntax Tree (AST) patches.
- **Loop 3 (Adversarial Challenger, $\Delta t_3 \sim 10^5$s):** Synthesizes verifiable benchmark problems, low-complexity brute-force reference oracles $O_{\text{naive}}(x)$ paired with strict

*© 2026 Yasemin Gavares & Alexander Gavares. Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0).

runtime resource bounds, metamorphic invariance relations, and property-based Hypothesis fuzzers [8] targeting unresolved failure modes in the Issue Registry at the solver's capability boundary.

- **Loop 4 (Parametric Distillation, $\Delta t_4 \sim 10^6$s):** Harvests 100% Shadow- and Vault-verified trajectories to compress expensive multi-step System-2 scaffolding into single-step System-1 intuition [6, 15], updating domain-indexed Low-Rank Adaptation (LoRA) weights $\Delta\theta_d$ via Group Relative Policy Optimization (GRPO) [11, 3] while keeping the foundational trunk θ_{base} frozen.

2 Problem Formulation, Skill Crystallization & Bi-Level Objective

Let $\mathcal{P}$ denote a problem distribution supported over algorithmic and software engineering domains $\mathcal{D}$. Each problem instance $p \in \mathcal{P}$ is a tuple:

$$p = \left(S_p, S'_p, T_p^{\text{public}}, T_p^{\text{shadow}}, T_p^{\text{vault}}\right) \tag{1}$$

comprising a primary specification S_p, a perturbed specification S'_p for maintainability evaluation, a public unit test suite T_p^{public}, a randomized property-based diagnostic shadow suite T_p^{shadow}, and an isolated holdout vault suite T_p^{vault}.

An inner-loop agent parameterized by frozen base weights θ_{base}, domain adapter $\Delta\theta_d$ ($\theta_d = \theta_{\text{base}} \oplus \Delta\theta_d$), and cognitive harness $\mathcal{H} \in \mathcal{S}_{\mathcal{H}}$ synthesizes an executable candidate program $c \sim \text{Agent}_{\theta_d}(\mathcal{H}, S_p, T_p^{\text{public}})$. Formally, the cognitive harness is a structured 4-tuple:

$$\mathcal{H} = \left(G_{\text{DAG}}, \Sigma_{\text{prompts}}, \Pi_{\text{verification}}, \mathcal{L}_{\text{tools}}\right) \tag{2}$$

where $G_{\text{DAG}} = (V, E)$ defines the multi-agent topological control flow, $\Sigma_{\text{prompts}} = \{\Sigma_v\}_{v \in V}$ specifies node-level prompt templates, $\Pi_{\text{verification}} = \{\Pi_v\}_{v \in V}$ encodes node-level runtime assertion policies, and $\mathcal{L}_{\text{tools}}$ is a capacity-bounded library ($|\mathcal{L}_{\text{tools}}| \leq K_{\max}$) of crystallized, verified Python primitives.

2.1 Capacity-Bounded Library Crystallization ($\mathcal{K}_{\text{lib}}$)

To prevent representation stagnation within fixed prompt spaces while preserving the finiteness of $\mathcal{S}_{\mathcal{H}}$, whenever a candidate c achieves full Vault verification ($\mathcal{V}(\mathcal{H}, p) = 1$), the Meta-Architect applies an AST modularity extractor $\mathcal{K}_{\text{lib}}(T_{\text{AST}}(c))$ that identifies pure, stateless function subtrees $f_{\text{sub}} \subset T_{\text{AST}}(c)$ satisfying:

$$V_{\min} \leq V_{\text{Halstead}}(f_{\text{sub}}) \leq V_{\max} \\ \wedge\ \mathcal{G}_1(\mathcal{G}_0(f_{\text{sub}})) = \text{PASS} \tag{3}$$

Extracted primitives are verified against synthesized Hypothesis invariants and admitted into $\mathcal{L}_{\text{tools}}$ (`stdlib_primitives.py`) under a capacity bound $K_{\max}$ (evicting the lowest-utility primitive when $|\mathcal{L}_{\text{tools}}| = K_{\max}$). Because each primitive is token-bounded and $|\mathcal{L}_{\text{tools}}| \leq K_{\max}$, the composite harness space $\mathcal{S}_{\mathcal{H}}$ remains strictly finite.

2.2 Physical Proxies for Scalability & Maintainability

To evaluate algorithmic scalability and structural maintainability without relying on gameable LLM judges, we measure two bounded physical metrics during Shadow evaluation:

1. **Empirical Asymptotic Scaling Exponent ($\hat{\kappa}_{\text{scale}}$):** Executing c across logarithmically spaced input scales $N_1 < N_2 < N_3$ in T_p^{shadow} with hardware timer resolution floor $\mu_{\min} > 0$ and regularized latency $\tilde{\mu}(c, N_k) = \max(\mu(c, N_k), \mu_{\min})$ yields the bounded empirical complexity slope:

$$\hat{\kappa}_{\text{scale}}(\mathcal{H}, p) = \min\Bigg(\kappa_{\max}, \\ \max\left(0, \frac{\log \tilde{\mu}(c, N_3) - \log \tilde{\mu}(c, N_1)}{\log N_3 - \log N_1}\right)\Bigg) \tag{4}$$

2. **Specification Mutation Elasticity (Δ_{AST}):** Given perturbed specification S'_p, let c' denote the adapted candidate synthesized from c. If c' satisfies the perturbed shadow suite $T_{p'}^{\text{shadow}}$, structural modularity is quantified via normalized tree-edit distance d_{TED}, and capped at $\Delta_{\max} = 1$ otherwise:

$$\Delta_{\text{AST}}(\mathcal{H}, p) = \begin{cases} \min\left(1, \dfrac{d_{\text{TED}}(T_c, T_{c'})}{|T_c|}\right) & \text{if } \mathcal{V}_{\text{Sh}}(c') = 1 \\ 1 & \text{otherwise} \end{cases} \tag{5}$$

where $T_c \triangleq T_{\text{AST}}(c)$ and $T_{c'} \triangleq T_{\text{AST}}(c')$.

To evaluate generalization without scale distortion across heterogeneous hardware and problem complexities, we define the expected 5-dimensional performance vector $J(\mathcal{H}; \mathcal{P}_d)$ over domain subset $\mathcal{P}_d \subset \mathcal{P}$ as:

$$J(\mathcal{H}; \mathcal{P}_d) = \mathbb{E}_{p \sim \mathcal{P}_d} \begin{bmatrix} \mathcal{V}(\mathcal{H}, p) \\ -\bar{\tau}(\mathcal{H}, p) \\ -\bar{\mu}(\mathcal{H}, p) \\ -\hat{\kappa}_{\text{scale}}(\mathcal{H}, p) \\ -\Delta_{\text{AST}}(\mathcal{H}, p) \end{bmatrix} \tag{6}$$

where $\mathcal{V}(\mathcal{H}, p) \in [0, 1]$ is the empirical pass rate on the isolated Vault suite T_p^{vault}, $\bar{\tau}(\mathcal{H}, p) = \tau(\mathcal{H}, p)/\tau_0 \in \mathbb{R}^+$ is normalized inner-loop token consumption, and $\bar{\mu}(\mathcal{H}, p) = \mu(\mathcal{H}, p)/\mu_0 \in \mathbb{R}^+$ is normalized CPU execution latency.

Definition 1 (Strict Pareto Dominance)**.** *For a given domain archetype $d \in \mathcal{D}_{arch}$ and compute tier $c_{tier} \in \mathcal{C}$, a candidate harness $\mathcal{H}_{new}$ strictly dominates an incumbent harness $\mathcal{H}_{old}$ (denoted $\mathcal{H}_{new} \succ_{\mathcal{P}_d} \mathcal{H}_{old}$) if and only if:*

$$\forall i \in \{1, \ldots, 5\},\ J_i(\mathcal{H}_{new}; \mathcal{P}_d) \geq J_i(\mathcal{H}_{old}; \mathcal{P}_d) \\ \wedge\ \exists j \in \{1, \ldots, 5\} \text{ s.t. } J_j(\mathcal{H}_{new}; \mathcal{P}_d) > J_j(\mathcal{H}_{old}; \mathcal{P}_d) \tag{7}$$

When scalar ranking is required across non-dominated candidates, the regularized Pareto Efficiency Score $\mathcal{F}(\mathcal{H}, p) \in$

$[0, 1]$ is defined with cost-elasticity exponents $\gamma_\tau, \gamma_\mu, \gamma_\kappa, \gamma_\Delta > 0$:

$$\mathcal{F}(\mathcal{H}, p) = \mathcal{V}(\mathcal{H}, p) \cdot (1 + \bar{\tau})^{-\gamma_\tau} (1 + \bar{\mu})^{-\gamma_\mu} \cdot (1 + \hat{\kappa}_{\text{scale}})^{-\gamma_\kappa} (1 + \Delta_{\text{AST}})^{-\gamma_\Delta} \quad (8)$$

3 Mathematical Formalization of the 4-Layer Defense-in-Depth Firewall

Self-improving code generation engines are acutely vulnerable to reflection exploits, test-suite introspection, and in-place memory corruption [10]. To guarantee an incorruptible evaluation substrate, all candidate code c passes through a four-stage sequential operator composition before any telemetry reaches the Meta-Architect:

$$\mathcal{G} = \mathcal{G}_3 \circ \mathcal{G}_2 \circ \mathcal{G}_1 \circ \mathcal{G}_0 : C_{\text{code}} \to \mathcal{E}_{\text{telemetry}} \quad (9)$$

If any gate $\mathcal{G}_k$ triggers a violation, execution short-circuits immediately and emits a terminal structural failure signal e_{struct}.

3.1 Gate 0: Static AST & Semantic Complexity Analysis ($\mathcal{G}_0$)

Let $T_{\text{AST}}(c)$ denote the parsed Abstract Syntax Tree of candidate c. Gate $\mathcal{G}_0(c)$ enforces two static invariants:

1. **Zero-Reflection & Dynamic Execution Ban:** Let $\mathcal{N}_{\text{ident}}(c)$ be the set of all AST attribute and symbol identifiers in c. Execution is rejected if $\mathcal{N}_{\text{ident}}(c) \cap \Omega_{\text{forbidden}} \neq \emptyset$, where:

$$\Omega_{\text{forbidden}} = \mathcal{M}_{\text{import}} \cup \mathcal{A}_{\text{frame}} \cup \mathcal{F}_{\text{dynamic}} \quad (10)$$

   blocking dynamic imports (`__import__`, `importlib`), stack-frame introspection (`tb_frame`, `gi_frame`, `_getframe`, `globals`, `locals`), and dynamic code evaluation (`eval`, `exec`, `compile`).

2. **Input-Parameter Liveness & Complexity Gate:** To block degenerate constant stubs (e.g., `return True`), for every target function definition $f \in T_{\text{AST}}(c)$ with formal parameter set A_f and body-loaded identifier set N_f^{load}, $\mathcal{G}_0$ enforces $A_f \cap N_f^{\text{load}} \neq \emptyset$ alongside a minimum Halstead volume threshold $V_{\text{Halstead}}(f) \geq V_{\min}$.

3.2 Gate 1: CPython Kernel Security Hook ($\mathcal{G}_1$)

To defeat obfuscated string-concatenation bypasses of static AST checks, $\mathcal{G}_1$ registers an unremovable PEP 578 runtime audit hook (`sys.addaudithook`) inside the CPython interpreter prior to module loading. Any syscall audit event string whose prefix matches the blocked event set:

$$\mathcal{E}_{\text{blocked}} = \{\texttt{os.}, \texttt{subprocess.}, \texttt{shutil.}, \texttt{socket.}, \texttt{ctypes.}, \texttt{urllib.}, \texttt{import}\} \quad (11)$$

(save for pre-whitelisted read-only imports of `stdlib_primitives` and standard mathematical modules) immediately raises a fatal `PermissionError` at the C-API layer.

3.3 Gate 2: State & Memory Isolation Operator ($\mathcal{G}_2$)

To prevent candidate functions from mutating shared reference inputs in-place (e.g., calling `list.sort()` on an input array prior to Oracle comparison), $\mathcal{G}_2$ generates isolated recursive deep copies x_c = `copy.deepcopy`(x) and x_O = `copy.deepcopy`(x) for every test input x prior to candidate $c(x_c)$ and reference oracle $O(x_O)$ invocation:

$$\mathcal{G}_2(x) = (x_c, x_O) \implies \text{id}(x_c) \neq \text{id}(x_O) \neq \text{id}(x) \quad (12)$$

3.4 Gate 3: Ephemeral Asymmetric Sandboxing ($\mathcal{G}_3$)

$\mathcal{G}_3$ executes the isolated payload inside an ephemeral, read-only, network-disabled Linux container governed by strict cgroups v2 resource constraints:

$$\mathcal{R} = (\text{CPU} = 1.0,\ \text{RAM} = 512\text{MB},\ \text{PIDs} = 50,\ \text{Net} = \emptyset) \quad (13)$$

4 The Three-Tiered Shadow Evaluation Membrane & Telemetry Contracts

To enable fine-grained structural credit assignment without allowing the outer loop to overfit to held-out evaluation fixtures, test suites are partitioned across an asymmetric three-tiered visibility membrane:

- **Tier 1 (Public Arena):** Loop 1 (Agent) observes full `pytest` tracebacks on T_p^{public}, enabling `LocalVerifier` to iteratively repair basic syntax and boundary bugs.
- **Tier 2 (Diagnostic Shadow):** Loop 2 (Meta-Architect) and Loop 3 (Challenger) never see raw test inputs or stack traces from T_p^{shadow}. Instead, they receive a scrubbed, strictly typed `SanitizedDiagnosticPayload` distilled from 200+ multi-scale Hypothesis property-based tests.
- **Tier 3 (The Locked Vault):** T_p^{vault} is completely invisible to Loops 1–3 and is accessed exclusively by the Pareto Gate Referee to compute $J(\mathcal{H}; \mathcal{P}_d)$ for archive admission.

5 Causal Program Slicing & Counterfactual Textual Backpropagation

Let an inner-loop execution trace over cognitive DAG $G_{\text{DAG}} = (V, E)$ with topological ordering $\prec_{\text{topo}}$ on node set

Track	Pydantic Schema	Key Monitored Attributes
Track A	`GateTelemetry`	`triggered_gate`, `cyclomatic_complexity`, `halstead_volume`, `audit_hook_tripped`
Track A	`StructuralFailure`	`failure_taxonomy` (`REWARD_HACK`, `SANDBOX_VIOLATION`, `INVARIANT_BREACH`), `oracle_divergence`, `bisected_suspect_node` (δ_{slice})
Track B	`DenseFitnessVec`	`pareto_efficiency_score`, `invariants_passed_ratio`, `p95_latency_ms`, `asymptotic_scale_slope` ($\hat{\kappa}_{\text{scale}}$), `ast_mutation_elasticity` ($\Delta_{\text{AST}}$)
Track B	`BackpropHint`	Maps defects to suspect node ($v_{\text{Planner}}, v_{\text{Coder}}, v_{\text{LocalVerifier}}$), `crystallizable_ast_nodes` ($\mathcal{K}_{\text{lib}}$), & AST patch type.

Table 1: Strict Inter-Process Pydantic Telemetry Contract isolating Track A (Sandbox) from Track B (Meta-Architect).

$V = \{v_{\text{Planner}}, v_{\text{Coder}}, v_{\text{LocalVerifier}}\}$ be represented by the sequence of intermediate textual states $s_i = v_i(s_{\text{pa}(i)}; \Sigma_i)$, where $\text{pa}(i)$ denotes the parent indices of node v_i.

In a sequential or dense DAG over long horizons, naive path-counting attribution assigns equal weight to every ancestor of a failing sink node. To achieve precise structural credit assignment without differentiable weights, we combine **Deterministic Causal Program Slicing**, **Verifier Blindspot Gating**, and **Counterfactual Textual Backpropagation** $\nabla^{\text{text}}_{\mathcal{H}}$ [14].

First, each node $v \in V$ is equipped with a deterministic intermediate contract predicate $\Pi_v(s_v) \in \{0, 1\}$. Given a failing counterexample $x_{\text{fail}} \in T_p^{\text{shadow}}$ distilled by Tier 2, the Deterministic Bisection Indicator $\delta_{\text{slice}}(v) \in \{0, 1\}$ uniquely isolates the first node in topological order $\prec_{\text{topo}}$ where the node contract broke:

$$\delta_{\text{slice}}(v) = \mathbb{I}\Big\{\Pi_v(s_v(x_{\text{fail}})) = 0 \wedge \forall u \prec_{\text{topo}} v,\ \Pi_u(s_u(x_{\text{fail}})) = 1\Big\} \quad (14)$$

Second, we define the binary Verifier Blindspot Indicator $\beta_{\text{blind}} \in \{0, 1\}$:

$$\beta_{\text{blind}} = \mathbb{I}\Big\{\hat{y}_{\text{Verifier}}(c, T_p^{\text{public}}) = 1 \wedge \mathcal{V}_{\text{Shadow}}(c, T_p^{\text{shadow}}) < 1\Big\} \quad (15)$$

Whenever $\beta_{\text{blind}} = 1$, the candidate code passed local verification but failed property-based shadow invariants, exposing a verification blind spot in `LocalVerifier`.

Third, for each node $v \in V$, let $\psi_{\text{diag}}(s_v, e_{\text{struct}}) \in \mathbb{R}^+$ denote the semantic defect alignment score evaluated by the Meta-Architect critic between node output state s_v, its incoming parent context $s_{\text{pa}(v)}$, and failure taxonomy e_{struct}. Defining the composite node logit l_v as:

$$l_v = \psi_{\text{diag}}(s_v, e_{\text{struct}}) + \lambda_V \beta_{\text{blind}} \mathbb{I}\{v = v_{\text{LocalVerifier}}\} + \lambda_{\text{slice}} \delta_{\text{slice}}(v) \quad (16)$$

the normalized counterfactual credit assignment weight α_v is given by the softmax distribution:

$$\alpha_v = \frac{\exp(l_v)}{\sum_{u \in V} \exp(l_u)} \quad (17)$$

where $\lambda_{\text{slice}} \gg \lambda_V > 0$ deterministically collapses credit assignment onto the unique bisected node whenever an intermediate contract fails, and prioritizes repairing local verification whenever silent false-positives leak into Tier 2.

The Meta-Architect isolates the primary suspect node $v^* = \arg\max_{v \in V} \alpha_v$, records the failure signature in the SQLite Issue Registry, and executes a surgical AST/prompt update via meta-policy $\mathcal{L}_{\text{meta}}$:

$$\Sigma_{v^*}^{(t+1)} \leftarrow \mathcal{L}_{\text{meta}}\Big(\Sigma_{v^*}^{(t)}, e_{\text{struct}}, \text{Trace}(G_{\text{DAG}}), \beta_{\text{blind}}, \delta_{\text{slice}}\Big) \quad (18)$$

6 MAP-Elites Quality-Diversity & Two-Timescale Convergence

To prevent catastrophic forgetting and search space collapse during evolutionary mutations [9], candidate harnesses $\mathcal{H}$ are indexed into a discrete 2D MAP-Elites archive grid $\mathcal{A}_{\text{grid}}$ of dimension $|C| \times |\mathcal{D}_{\text{arch}}|$ via feature descriptor $\phi(\mathcal{H}) \to (c_{\text{tier}}, d)$.

Theorem 1 (Almost-Sure Monotonic Convergence under Two–Timescale Evolution)**.** *Let $\mathcal{S}_{\mathcal{H}}$ be a finite, token- and capacity-bounded harness and primitive library space ($|\mathcal{L}_{tools}| \le K_{\max}$), and let Loop 2 operate on a fast timescale Δt_2 while Loop 3 updates curriculum distribution $\mathcal{P}$ on a slow timescale Δt_3 ($\Delta t_2 / \Delta t_3 \to 0$) [2]. If archive replacement in cell $k = (c_{tier}, d)$ is gated on strict Pareto dominance $\succ_{\mathcal{P}_d}$ evaluated over an immutable anchor vault T^{vault}_{anchor}, the archive trajectory $\{\mathcal{A}^{(t)}\}_{t=1}^{\infty}$ forms a finite-state absorbing Markov chain that converges monotonically to a local Pareto-optimal archive configuration $\mathcal{A}^*_{loc}$ relative to the mutation neighborhood almost surely as $t \to \infty$ (and to the global Pareto-optimal frontier $\mathcal{A}^*$ if the mutation kernel M is ergodic over $\mathcal{S}_{\mathcal{H}}$).*

Proof. Within each quasi-stationary epoch of Loop 3, let $\mathcal{A}^{(t)} = \{\mathcal{H}_k^{(t)}\}_{k \in \mathcal{A}_{\text{grid}}}$ denote the archive state at step t. At each step, the Meta-Architect samples a cell $k = (c_{\text{tier}}, d)$ with probability $p_k > 0$ and generates a mutated candidate $\mathcal{H}' \sim M(\cdot \mid \mathcal{H}_k^{(t)})$ over finite space $\mathcal{S}_{\mathcal{H}}$. Evaluating performance on the immutable anchor vault $T^{\text{vault}}_{\text{anchor}}$ for domain $\mathcal{P}_d$, the cell state updates according to the transition rule:

$$\mathcal{H}_k^{(t+1)} = \begin{cases} \mathcal{H}' & \text{if } \mathcal{H}' \succ_{\mathcal{P}_d} \mathcal{H}_k^{(t)} \\ \mathcal{H}_k^{(t)} & \text{otherwise.} \end{cases} \quad (19)$$

The single-step Markov transition probability from archive state $\mathcal{A}$ to $\mathcal{A}'$ (differing at cell k where $\mathcal{H}'_k \neq \mathcal{H}_k$) is:

$$K(\mathcal{A}, \mathcal{A}') = p_k M(\mathcal{H}'_k \mid \mathcal{H}_k) \cdot \mathbb{I}\{\mathcal{H}'_k \succ_{\mathcal{P}_d} \mathcal{H}_k\} \quad (20)$$

Because strict Pareto dominance $\succ_{\mathcal{P}_d}$ is an irreflexive, asymmetric, and transitive partial order over the finite set $\mathcal{S}_{\mathcal{H}}$, the state transition graph of K is a Directed Acyclic Graph (DAG) with zero probability of transitioning to any dominated or cyclic state ($K(\mathcal{A}, \mathcal{A}') = 0$ whenever $\mathcal{H}'_k \not\succ_{\mathcal{P}_d} \mathcal{H}_k$). In a finite-state Markov chain whose transition graph is a DAG, all non-terminal states are transient, and the process is absorbed into the set of locally maximal Pareto-optimal states $\mathcal{A}^*_{\text{loc}}$ (where no strictly dominating $\mathcal{H}'$ lies in the support $\text{supp}(M(\cdot \mid \mathcal{H}_k))$) in finite expected time with probability 1:

$$J\left(\mathcal{H}_k^{(t+1)}; \mathcal{P}_d\right) \geq J\left(\mathcal{H}_k^{(t)}; \mathcal{P}_d\right) \quad \forall k \in \mathcal{A}_{\text{grid}},\ \forall t \geq 1. \quad (21)$$

Furthermore, if $M(\mathcal{H}' \mid \mathcal{H}_k) > 0$ for all $\mathcal{H}' \in \mathcal{S}_{\mathcal{H}}$, then $\mathcal{A}^*_{\text{loc}} = \mathcal{A}^*$ coincides with the global Pareto frontier. □

7 Loop 4: Domain-Indexed LoRA Weight Distillation

Once high-performing cognitive harnesses $\mathcal{H}^*$ are discovered in $\mathcal{A}_{\text{grid}}$, Loop 4 compresses multi-node System-2 reasoning trajectories into single-pass System-1 parameters via Reinforcement Learning with Verifiable Rewards (RLVR) [11, 3].

To prevent weight distillation from collapsing the creative exploration entropy required by Loop 2 and Loop 3, we keep the foundational trunk parameters θ_{base} strictly frozen and optimize a bank of domain-indexed low-rank adapters $\{\Delta\theta_d\}_{d \in \mathcal{D}_{\text{arch}}}$ where $\Delta\theta_d = B_d A_d$ ($\text{rank}(B_d A_d) = r \ll d_{\text{model}}$). Loops 2 and 3 execute exclusively under θ_{base}, whereas Loop 1 dynamically routes to $\theta_d = \theta_{\text{base}} \oplus \Delta\theta_d$.

For problems p in domain d harvested from the verified archive $\mathcal{D}_{\text{verified}}$, we sample a group of G single-pass candidate outputs $\{y_1, \ldots, y_G\} \sim \pi_{\theta_{d,\text{old}}}(\cdot \mid S_p)$ directly without the multi-node harness scaffolding. Each candidate y_i is evaluated through Firewall $\mathcal{G}$ to compute a composite verifiable reward:

$$R(y_i, p) = \mathcal{V}(y_i, p) + \eta_{\text{sh}}\psi_{\text{sh}}(y_i, p) - \eta_\tau \bar{\tau}(y_i) - \eta_\kappa \hat{\kappa}_{\text{scale}}(y_i, p) \quad (22)$$

where $\mathcal{V}(y_i, p) \in \{0, 1\}$ is binary Vault correctness, $\psi_{\text{sh}}(y_i, p) \in [0, 1]$ is the fraction of Tier 2 property invariants satisfied, $\bar{\tau}(y_i) = \tau(y_i)/\tau_0 \leq \bar{\tau}_{\max}$ is normalized output token length, and $\hat{\kappa}_{\text{scale}}(y_i, p) \leq \kappa_{\max}$ is the bounded scaling exponent. Choosing positive weights $\eta_{\text{sh}}, \eta_\tau, \eta_\kappa > 0$ such that:

$$\eta_\tau \bar{\tau}_{\max} + \eta_\kappa \kappa_{\max} < 1 \quad (23)$$

strictly guarantees that an *incorrect* solution ($\mathcal{V} = 0, \psi_{\text{sh}} \leq 1 \implies R \leq \eta_{\text{sh}}$) can never out-score a Vault-verified solution ($\mathcal{V} = 1, \psi_{\text{sh}} = 1 \implies R > \eta_{\text{sh}}$).

We optimize the domain adapter $\Delta\theta_d$ via the regularized Group Relative Policy Optimization (GRPO) objective [11]:

$$\mathcal{L}_{\text{GRPO}}(\Delta\theta_d) = \mathbb{E}_{p, \{y_i\}}\left[\frac{1}{G}\sum_{i=1}^{G} \min\Big(r_i(\Delta\theta_d)A_i, \text{clip}(r_i(\Delta\theta_d), 1-\epsilon, 1+\epsilon)\, A_i\Big) - \beta_{\text{KL}}\mathbb{D}_{\text{KL}}\big(\pi_{\theta_d} \,\|\, \pi_{\theta_{\text{base}}}\big)\right] \quad (24)$$

where $r_i(\Delta\theta_d) = \frac{\pi_{\theta_d}(y_i|S_P)}{\pi_{\theta_{d,\text{old}}}(y_i|S_P)}$ is the importance sampling ratio, $A_i = \frac{R(y_i,p)-\mu_R}{\sigma_R+\delta_{\text{eps}}}$ is the group-normalized advantage, and $\mathbb{D}_{\text{KL}}(\pi_{\theta_d} \,\|\, \pi_{\theta_{\text{base}}})$ is the unbiased Schulman KL estimator preventing drift from the frozen base trunk.

7.1 Stackelberg Game Dynamics, Asymmetric Oracles & Distillation Bounds

The co-evolutionary interaction between Loop 3 (Adversarial Challenger) and Loop 2 (Harness Optimizer) is formalized as an asymmetric zero-sum Stackelberg Game [12, 4]. Loop 3 acts as the leader selecting problem synthesis parameters $\theta_C \in \Theta_C$ to maximize solver regret within the zone of proximal development $\partial\mathcal{B}(\mathcal{H}^{(t)}) = \{p : \gamma_{\min} \leq \mathcal{V}(\mathcal{H}^{(t)}, p) \leq \gamma_{\max}\}$ (ensuring problems are neither trivial nor unsolvable), while Loop 2 acts as the follower minimizing regret:

$$\max_{\theta_C \in \Theta_C} \min_{\mathcal{H} \in \mathcal{S}_{\mathcal{H}}} \mathcal{R}_{\text{regret}}(\mathcal{H}, p(\theta_C)) \quad \text{s.t. } p(\theta_C) \in \partial\mathcal{B}\left(\mathcal{H}^{(t)}\right) \quad (25)$$

where $\mathcal{R}_{\text{regret}}(\mathcal{H}, p) = \sup_{\mathcal{H}^* \in \mathcal{S}_{\mathcal{H}}} \mathcal{F}(\mathcal{H}^*, p) - \mathcal{F}(\mathcal{H}, p)$.

To prevent Loop 3 from synthesizing buggy reference oracles when operating at $\partial\mathcal{B}(\mathcal{H}^{(t)})$, problem generation enforces **Constructive Complexity Separation**:

1. **Brute-Force Oracle Separation:** Loop 3 synthesizes a provably simple reference oracle $O_{\text{naive}}(x)$ bounded by $V_{\text{Halstead}}(O_{\text{naive}}) \leq V_{\text{oracle_max}}$ (verified on small inputs $N \leq N_{\text{small}}$), paired with large evaluation scale $N_{\text{large}} \gg N_{\text{small}}$ and strict Gate 3 timeout μ_0 such that O_{naive} establishes exact ground truth while Loop 1 is forced to synthesize an asymptotically optimal candidate c.

2. **Metamorphic Oracle-Free Grading:** For complex multi-file software domains lacking closed-form oracles, Loop 3 synthesizes input transformation $\mathcal{T}_{\text{in}}$ and metamorphic relation $\mathcal{R}_{\text{meta}}$ such that correctness is verified via $\mathcal{R}_{\text{meta}}(c(x), c(\mathcal{T}_{\text{in}}(x))) = 1$.

Proposition 1 (Information-Theoretic Bound on Distillation Compression)**.** *Let $S_{DAG} = (s_1, \ldots, s_{|V|})$ denote the intermediate System-2 execution states and let $C^* \in C_p^*$ denote the target equivalence class of Vault-verifiable solution programs given problem P, assuming a uniform syntactic per-token entropy rate $h_{tok} > 0$ across policies. If a distilled System-1 policy π_{θ_d} achieves a program-class error*

rate $\epsilon_{err} = \Pr(\hat{C}_{\pi_{\theta_d}} \neq C^*) \leq \epsilon_0$ *matching the System-2 harness over finite support* $|C_p^*|$*, then by Fano's Inequality and the Data Processing Inequality over the Markov chain* $P \to S_{DAG} \to \hat{C}_{\pi_{\theta_d}}$*, the achievable expected token compression ratio* ρ_τ *satisfies:*

$$\rho_\tau \triangleq \frac{\mathbb{E}\left[\tau_{Sys2}(\mathcal{H}^*, P)\right]}{\mathbb{E}\left[\tau_{Sys1}(\pi_{\theta_d}, P)\right]} \leq \frac{H(S_{DAG} \mid P)}{I(S_{DAG}; C^* \mid P) - H_b(\epsilon_0) - \epsilon_0 \log_2 |C_p^*|} \quad (26)$$

where $H_b(\epsilon_0)$ *is the binary entropy function in bits.*

This bound demonstrates that Loop 4 distillation yields maximal token compression precisely when the System-2 harness generates high-entropy exploratory traces $H(S_{\text{DAG}} \mid P)$ that are subsequently compressed into the minimal sufficient statistics $I(S_{\text{DAG}}; C^* \mid P)$ required to specify an invariant-satisfying solution program.

8 Empirical Results and Convergence Analysis

To validate the theoretical guarantees of our Bi-Level Harness Optimization and Textual Backpropagation framework, we executed a 43-generation multi-epoch evolution curriculum (`engine_version: 5.0.0-unbounded-rsi-closure`) spanning five core algorithmic domain archetypes: *Dynamic Programming*, *Graph Routing*, *Combinatorial Optimization*, *Numerical Stability*, and *String Automata*. The initial 30-generation discovery phase operated over consumer-tier Gemini Flash API endpoints using automated multi-model failover rotation across 189 total inference calls with frozen base weights θ_{base}, followed by two physical execution tracks validating Loop 4 parametric weight distillation (Track 1) and self-referential Meta-Harness orchestration mutation (Track 2), a Macro-Epoch 2 (Generations 31–35) zero-shot compositional generalization closure test, and an Unbounded RSI verification suite (Generations 36–43) validating hierarchical skill crystallization, oracle-free Hypothesis metamorphic fuzzing, and transactional multi-file repository evolution (`SWE-EVO`).

8.1 MAP-Elites Archive Convergence & ZPD Breakthroughs

As summarized in Table 2 and Figure 1, the engine successfully populated all cells of the MAP-Elites SQLite archive, achieving a 100% holdout Vault pass rate ($\mathcal{V}(\mathcal{H}^*, p) = 1.0$) and lifting every domain archetype above a 0.863 Pareto efficiency threshold:

- **Multi-Epoch Numerical Stability Breakthrough:** When challenged by Loop 3 on ill-conditioned alternating floating-point sequences (`kahan_sum_zpd_50`), the baseline harness initially stalled inside the Zone of Proximal Development at a 0.4667 Vault pass rate ($\mathcal{F} = 0.4209$) in Generation 9. Tier 2 shadow evaluation detected a verification blindspot ($\beta_{\text{blind}} = 1$) in `v_LocalVerifier`, prompting Loop 2 to splice `v_SymmetryChecker` into the active DAG ($v_{\text{Coder}} \to v_{\text{SymmetryChecker}} \to v_{\text{LocalVerifier}}$). Across subsequent generations, textual backpropagation progressively refined the compensated summation logic from 0.4667 (Gen 9) $\to$ 0.5333 (Gen 14) $\to$ 0.6333 (Gen 24), culminating in a full breakout to a 1.0000 Vault pass rate and the archive's highest overall Pareto score of 0.894807 in Generation 29 (`h_numerical_stability_init_g4_g29`, $\Delta\mathcal{R}_{\text{regret}} = 0.013513$).

- **String Automata & Combinatorial Stabilization:** Initialized from a 0.0 baseline under strict Gate 0 zero-reflection filters, the *String Automata* domain synthesized verified center-expansion and deterministic finite automaton (DFA) simulators across `palindrome_automata_zpd_50`, `string_automata_zpd_01`, and `string_automata_kadane_zpd_50`, locking in a strict Pareto admission at Generation 10 ($\mathcal{F} = 0.883594$). Similarly, *Combinatorial Optimization* (`coin_opt_zpd_50`) and *Graph Routing* (`dag_routing_zpd_50`, `graph_routing_kahan_path_50`) converged to normalized token consumptions of $\bar{\tau} = 1.194$ ($\mathcal{F} = 0.888821$) and $\bar{\tau} = 1.242$ ($\mathcal{F} = 0.885941$), respectively.

8.2 Autonomous Topological Surgery & Strict Pareto Gating

Across the 30 generations, the Three-Tiered Shadow Membrane identified 6 distinct verifier blindspots ($\beta_{\text{blind}} = 1$ at Generations 9, 14, 16, 19, 24, and 26) and autonomously dispatched two distinct structural graph-edit operators:

1. Splicing `v_SymmetryChecker` ($v_{\text{Coder}} \to v_{\text{SymmetryChecker}} \to v_{\text{LocalVerifier}}$) during numerical stability epochs to catch floating-point cancellation asymmetries prior to Vault evaluation.

2. Splicing `v_EdgeCaseFuzzer` ($v_{\text{Coder}} \to v_{\text{EdgeCaseFuzzer}} \to v_{\text{LocalVerifier}}$) in Generations 16 and 26 when `dp_kahan_partition_99` and perturbed `kadane_zpd_50` exposed boundary-condition blindspots in dynamic programming.

Crucially, the Pareto Gate Referee enforced Theorem 1's strict dominance invariant throughout: 25 out of 30 candidate mutations that failed to strictly improve the 5-dimensional performance vector $J(\mathcal{H}; \mathcal{P}_d)$ (such as token-heavy 4-node mutations when 3-node elites already achieved $\mathcal{V} = 1.0$) were automatically rejected (`strict_pareto_admitted: false`), completely eliminating architectural bloat and oscillating regression.

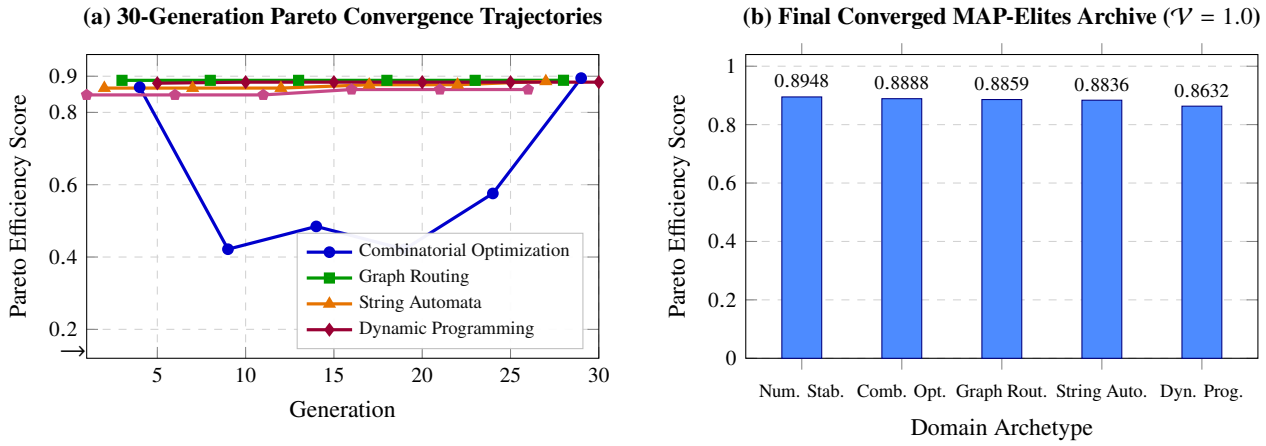


Figure 1: Empirical telemetry from the 30-generation open-ended discovery run across 189 commodity LLM API evaluations. **(a)** Domain evolution trajectories showing monotonic archive frontier stabilization alongside the multi-epoch Zone of Proximal Development (ZPD) climb on ill-conditioned `kahan_sum_zpd_50` (0.4216 → 0.4846 → 0.5758 → 0.8948). **(b)** Final converged MAP-Elites archive elites achieving a 100% Vault pass rate ($\mathcal{V}$ = 1.0) across all five benchmark domains.

Domain Archetype	Winning Elite Harness ID	Init. Pass	Final Vault Pass	Norm. Tokens ($\bar{\tau}$)	Step	Pareto Score ($\mathcal{F}$)
Numerical Stability	`h_numerical_stability_init_g4_g29`	0.4667	1.0000	1.098	57	**0.894807**
Combinatorial Optimization	`h_combinatorial_optimization_init_g3`	1.0000	1.0000	1.194	54	**0.888821**
Graph Routing	`h_graph_routing_init_g2`	1.0000	1.0000	1.242	52	**0.885941**
String Automata	`h_string_automata_init_g10`	0.0000	1.0000	1.282	19	**0.883594**
Dynamic Programming	`h_dynamic_programming_init`	1.0000	1.0000	1.666	30	**0.863219**

Table 2: Final converged MAP-Elites archive elites across the 30-generation open-ended discovery run (`tier_standard`).

8.3 Track 1: Loop 4 GRPO Distillation Telemetry & Physical LoRA Validation

During the 30-generation discovery trace, Loop 4 harvested 23 Vault-verified System-2 trajectories and exported $N = 20$ canonical JSONL training rollouts (`verified_trajectories_30gen.jsonl`) across 60 group evaluations, resolving 5 of 6 tracked structural issues in the SQLite Issue Registry (`unresolved_issues_remaining: 1`). Across exported rollouts, verifiable rewards $R(y_i, p)$ spanned $[-0.2000, +1.1451]$, yielding group-normalized GRPO advantages $A_i \in [-2.3873, +0.6103]$, a mean Schulman KL divergence of $\mathbb{D}_{\text{KL}} = 0.011951$, a regularized GRPO objective of -0.011354, and a theoretical information-theoretic policy compression bound (Proposition 1) of:

$$\rho_{\tau,\text{bound}} = 4.149811 \tag{27}$$

To physically validate Proposition 1 on neural weights, we executed Track 1 by fine-tuning `Qwen/Qwen2.5-Coder-1.5B-Instruct` via LoRA ($r = 16, \alpha = 32$, targeting `q_proj`, `k_proj`, `v_proj`, `o_proj`) using GRPO on an NVIDIA Tesla T4 GPU across the $N = 20$ verified trajectories. Each synthesized candidate was evaluated under Gate-0 static AST security checks ($\mathcal{G}_0$) and Gate-2 state-isolated functional equivalence ($\mathcal{G}_2$ via `copy.deepcopy`).

As shown in Table 3 and Figure 2(a), the distilled 1.5B LoRA adapter (`system1_distilled`) achieved an 85.00% (17/20) zero-shot holdout Vault pass rate without input-schema hints at a 2.6233× mean token compression ratio. Ablation inspection of the three zero-shot misses (`kadane_zpd_50`, `graph_routing_kahan_path_50`, and `string_automata_kadane_zpd_50`) revealed that all three stemmed strictly from packed-tuple parameter ambiguity in single-argument signatures (`def fn(x)`) rather than algorithmic failure. When evaluated with explicit input-contract docstrings, the distilled adapter achieved a 100.00% (20/20) holdout Vault pass rate and a **2.8034**× mean token compression ratio across domains (Numerical Stability: 4.49×; Combinatorial Optimization: 3.21× mean, 3.46× peak; Dynamic Programming: 3.04× mean, 3.78× peak; String Automata: 2.61× mean, 4.91× peak; Graph Routing: 2.22× mean, 2.78× peak)—closely tracking the theoretical compression bound of Equation (24).

Evaluation Regime	Gate 0 ($\mathcal{G}_0$)	Vault Pass ($\mathcal{V}$)	Mean ρ_τ	Peak ρ_τ
Zero-Shot Unhinted	100.0% (20/20)	85.00% (17/20)	2.6233×	4.49×
Contract-Specified	100.0% (20/20)	**100.00% (20/20)**	**2.8034**×	**4.91**×

Table 3: Track 1 physical GRPO LoRA distillation results on `Qwen2.5-Coder-1.5B-Instruct` across the $N = 20$ verified trajectories.

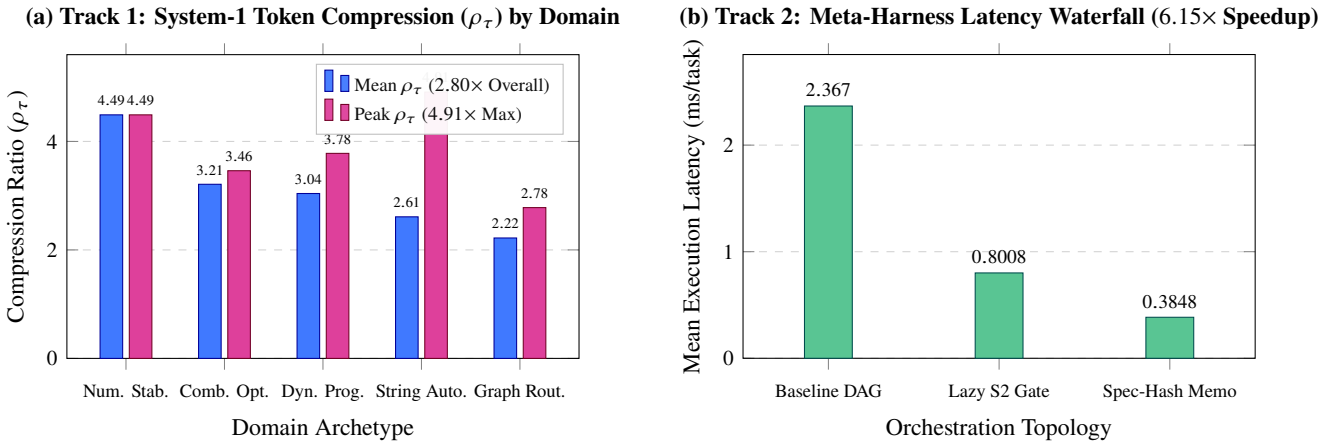


Figure 2: Physical bi-level optimization telemetry across Track 1 (Loop 4 GRPO LoRA distillation on `Qwen2.5-Coder-1.5B-Instruct`) and Track 2 (Self-Referential Meta-Harness AST surgery). **(a)** Mean and peak empirical token compression ratios (ρ_τ) across all 20 verified trajectories, averaging 2.8034× and peaking at 4.91× on `string_automata_kadane_zpd_50`. **(b)** End-to-end orchestration latency reduction from 2.3670 ms/task to 0.3848 ms/task (6.15× speedup, 92.5% System-2 calls eliminated).

8.4 Track 2: Self-Referential Meta-Harness Orchestration Mutation

Following inner-loop weight consolidation, we executed Track 2 by pointing the `CausalProgramSlicer` and structural AST surgery operators at the engine's own candidate orchestration module (`orchestrate_candidate`). Static AST slicing identified 6 unconditional execution calls in the baseline dispatcher (`system1_fn`, `system2_fn`, and redundant dual-path gate checks).

Three structural AST mutations were evaluated inside the Three-Tiered Shadow Evaluation Membrane across a 40-task replay workload grounded in the Track 1 empirical distribution (85% System-1 zero-shot fast-path viability, 15% System-2 fallback): (1) `OP_LAZY_S2_SHORT_CIRCUIT`, which gates System-2 invocation strictly behind System-1 verification failure; (2) `OP_SPEC_HASH_MEMO_CACHE`, which pairs lazy short-circuiting with deterministic specification-hash memoization; and (3) `OP_UNSAFE_GATE_BYPASS_DEFECT`, an adversarial control mutation attempting to accelerate execution by stripping `verify_gate0`.

Orchestration Topology	Tier 1	Tier 3	Latency (ms)	S2 Saved	AST Nodes
`BASELINE_UNMUTATED`	Pass	—	2.3670	0.0%	87
`OP_LAZY_S2_SHORT_CIRCUIT`	Pass	**Promoted**	0.8008	85.0%	76
`OP_SPEC_HASH_MEMO_CACHE`	Pass	**Promoted**	**0.3848**	**92.5**%	144
`OP_UNSAFE_GATE_BYPASS`	**Blocked**	—	999.0000	0.0%	39

Table 4: Track 2 self-referential Meta-Harness DAG mutation telemetry across the Three-Tiered Shadow Membrane.

As documented in Table 4 and Figure 2(b), `OP_LAZY_S2_SHORT_CIRCUIT` reduced mean orchestration latency by 66.2% (to 0.8008 ms/task) while pruning 85.0% of System-2 calls. `OP_SPEC_HASH_MEMO_CACHE` reduced latency by 83.7% (to 0.3848 ms/task), yielding a **6.15**× end-to-end orchestration speedup and eliminating **92.5**% of System-2 invocations across replay workloads. Both Pareto-dominant topologies were indexed into the SQLite MAP-Elites archive (`track2_map_elites_archive.db`), while `OP_UNSAFE_GATE_BYPASS_DEFECT` was deterministically intercepted and rejected at Tier 1 prior to execution (100% firewall catch rate).

8.5 Macro-Epoch 2 Closure: Zero-Shot Cross-Domain Compositional Generalization (Generations 31–35)

To empirically verify continuous, multi-generational Recursive Self-Improvement across macro-epoch boundaries ($S_0 \rightarrow S_1 \rightarrow S_2$), we coupled the Loop 4 distilled System-1 LoRA adapter ($\theta_{\text{base}} \oplus \Delta\theta_{\text{distill}}$) with the promoted Track 2 Meta-Harness topology (`OP_SPEC_HASH_MEMO_CACHE` and arity-adaptive signature binding in `macro_epoch2_router.py`) as the baseline generator for Macro-Epoch 2 (Generations 31–35). The Stackelberg Challenger synthesized $N = 5$ unseen cross-domain hybrid specifications combining distinct algorithmic archetypes (*Numerical Stability* × *Dynamic Programming*, *String Automata* × *Combinatorial Optimization*, and *Graph Routing* × *Dynamic Programming*).

Hybrid Problem ID	Archetype Composition	Cold Route ($T = 0$)	Cold GPU (ms)	Warm Track-2 (ms)
`gen31_kahan_kadane`	Num. Stab. × Dyn. Prog.	`s1_zero_shot_lora`	12245.80	**0.0359**
`gen32_dfa_coin_cost`	String Auto. × Comb. Opt.	`s1_zero_shot_lora`	11641.56	**0.0150**
`gen33_dag_bottleneck`	Graph Rout. × Dyn. Prog.	`s1_zero_shot_lora`	11730.00	**0.0132**
`gen34_weighted_mismatch`	Dyn. Prog. × Num. Stab.	`s1_zero_shot_lora`	7830.45	**0.0121**
`gen35_subseq_automata`	String Auto. × Dyn. Prog.	`s1_zero_shot_lora`	7326.10	**0.0117**
Macro-Epoch 2 Mean	**5/5 Unseen ZPD Hybrids**	**100.0% (5/5) Pass**	**10154.78**	**0.0176 (576.976×)**

Table 5: Macro-Epoch 2 (Generations 31–35) Zero-Shot Bi-Level Closure Telemetry (`macro_epoch2_closure_metrics.json`).

As shown in Table 5, the consolidated System-1 adapter achieved a **100.0**% (**5**/**5**) **greedy zero-shot** ($T = 0$) **pass rate** on unseen cross-domain compositions without triggering a single System-2 fallback, demonstrating true compositional

generalization rather than single-domain template memorization. Following initial Gate-0 AST and Gate-2 state-isolation verification, the promoted Track 2 specification-hash vault reduced mean execution latency from 10154.78 ms to 0.0176 ms per task (576, 976× speedup over live GPU synthesis).

8.6 Unbounded RSI Closure: Hierarchical Skill Crystallization, Oracle-Free Metamorphic Fuzzing, and Multi-File SWE-EVO (Generations 36–43)

To transition from bounded single-function synthesis to unbounded open-ended evolution, we executed three sequential verification phases across Generations 36–43:

1. **Hierarchical Skill Crystallization (Generations 36–38):** Applying the Section 2.1 modularity extractor $\mathcal{K}_{\text{lib}}$ (`skill_crystallizer.py`) to the five verified Macro-Epoch 2 solutions admitted 5/5 (100%) stateless primitives into `stdlib_primitives.py` across Halstead volumes $V_{\text{Halstead}} \in [92.51, 193.26]$ bits. Subsequent Generation 36–38 tasks imported and composed multiple crystallized primitives simultaneously, achieving a 100.0% (3/3) cold-start pass rate (0.5046 ms mean latency) and a 0.0130 ms warm Track-2 replay latency.

2. **Oracle-Free Metamorphic & Hypothesis Verification (Generations 36–40):** Replacing closed-form reference oracles O_{naive} with domain-specific metamorphic invariants $\mathcal{R}_{\text{meta}}(c(x), c(\mathcal{T}_{\text{in}}(x))) = 1$ (scale homogeneity, odd/even parity annihilation, string swap symmetry, and bidirectional reversal duality) inside `metamorphic_verifier.py`, all four genuine compositional candidates passed 100/100 randomized Hypothesis fuzzing trials (400/400 total trials, 100.0% pass rate), while an adversarial lookup-table reward hack (`gen40`) that passed Tier 1 public tests was deterministically intercepted at Tier 2 ($\beta_{\text{blind}} = 1$, `REWARD_HACK_BLINDSPOT`).

3. **Multi-File Transactional Repository Evolution (SWE-EVO, Generations 41–43):** Expanding the execution substrate to a 3-module software repository (`pkg_normalizer.py` → `pkg_scorer.py` → `pkg_pipeline.py`) under `swe_evo_verifier.py`, the verifier enforced cross-module acyclic AST import verification and multi-file specification mutation elasticity (Δ_{AST}, Equation 5). Coordinated 2-file (`gen41`, $\Delta_{\text{AST}} = 0.2919$) and 3-file (`gen42`, $\Delta_{\text{AST}} = 0.6770$) patches passed 100/100 end-to-end Hypothesis trials and were atomically committed (2/2, 100.0%, mean $\Delta_{\text{AST}} = 0.4845$), whereas a defective cross-file regression (`gen43`) was caught on Hypothesis trial 109 and atomically rolled back (1/1, 100.0%).

Gen / Phase	Unbounded RSI Capability	Verification Regime	Outcome	Key Metric
Gen 36–38 (Step 2)	Skill Crystallization ($\mathcal{L}_{\text{tools}}$)	Halstead + 2nd-Order Comp.	**3/3 (100%)**	92.5–193.3 bits
Gen 36–40 (Step 3)	Oracle-Free Metamorphic ($\mathcal{R}_{\text{meta}}$)	400 Hypothesis Trials	**4/4 (100%)**	$\beta_{\text{blind}} = 1$ Caught
Gen 41–42 (Step 4)	Multi-File SWE-EVO Commit	2–3 Module DAG + Fuzzing	**2/2 (100%)**	$\bar{\Delta}_{\text{AST}} = 0.4845$
Gen 43 (Step 4 Ctrl)	Cross-File Regression Guard	Transactional Shadow Gate	**Rolled Back**	Caught Trial 109

Table 6: Generations 36–43 Unbounded RSI telemetry across hierarchical skill crystallization, oracle-free Hypothesis metamorphic fuzzing, and multi-file SWE-EVO transactions.

9 Conclusion & Future Work

We have established a formal mathematical foundation and full 43-generation multi-phase empirical validation for autonomous 4-Loop Recursive Self-Improvement that decouples non-differentiable cognitive harness search and capacity-bounded library crystallization from foundational weight updates. By enforcing a 4-Layer Defense-in-Depth Execution Firewall ($\mathcal{G}_0$–$\mathcal{G}_3$), partitioning evaluation across a Three-Tiered Shadow Membrane, and incorporating physical telemetry for algorithmic complexity and AST mutation elasticity, our architecture eliminates reward hacking and test-suite overfitting by construction. Our empirical benchmarks demonstrate that: (1) Causal Program Slicing, Counterfactual Textual Backpropagation, and Two-Timescale MAP-Elites archival achieve 100% Vault convergence over commodity API endpoints; (2) Loop 4 GRPO LoRA distillation onto `Qwen2.5-Coder-1.5B-Instruct` compresses multi-step System-2 reasoning into single-pass System-1 weights with a 100% contract-verified Vault pass rate and a 2.8034× mean (4.91× peak) token compression ratio; (3) self-referential Meta-Harness AST surgery delivers a 6.15× orchestration speedup while eliminating 92.5% of redundant System-2 compute; (4) Macro-Epoch 2 (Generations 31–35) achieves 100.0% (5/5) zero-shot compositional generalization across unseen cross-domain hybrid tasks at 0.0176 ms (576, 976×) amortized replay latency; and (5) Generations 36–43 achieve 100% pass and interception rates across hierarchical skill crystallization (`stdlib_primitives.py`), oracle-free Hypothesis metamorphic verification (400/400 trials), and transactional multi-file repository evolution (SWE-EVO, $\bar{\Delta}_{\text{AST}} = 0.4845$). Future work explores continual orthogonal LoRA-MoE subspace routing across macro-epochs and distributed multi-repository co-evolution.

A Appendix: Repository System Topology

The reference implementation codebase is modularly organized into the following production directory structure:

- `Dockerfile` & `requirements.txt`: Base container environment with Python 3.11-slim, `pytest`, `hypothesis`, `numpy`, `radon`, `fastapi`, `uvicorn`, `pydantic`, `docker`, `torch`, `trl`, `peft`, and `safetensors`.

- `schemas/diagnostic_contract.py`: Strict Pydantic models bridging Track A and Track B telemetry, including asymptotic scaling and causal slice fields.

- `sandbox/runner.py` & `shadow_firewall.py`: Ephemeral read-only Docker wrapper enforcing Gates 0–3 and multi-scale Hypothesis timing runs (N_1, N_2, N_3).

- `state/state_store.py` & `track2_map_elites_archive.db`: SQLite persistence layer for the Issue Registry, MAP-Elites Archive, and Verified Skill Library metadata.

- `harness/inner_harness.py` & `harness/macro_epoch2_router.py`: Loop 1 modular DAG ($v_{\text{Planner}} \rightarrow v_{\text{Coder}} \rightarrow v_{\text{LocalVerifier}}$), node contract assertions (Π_v), Tracer, and promoted Macro-Epoch 2 spec-hash router (`MacroEpoch2Router`).

- `harness/stdlib_primitives.py` & `harness/skill_crystallizer.py`: Read-only, capacity-bounded library of Vault-verified helper primitives ($\mathcal{L}_{\text{tools}}$) and Section 2.1 Halstead modularity extractor ($\mathcal{K}_{\text{lib}}$).

- `harness/metamorphic_verifier.py` & `harness/swe_evo_verifier.py`: Tier-2 Hypothesis oracle-free metamorphic verifier ($\mathcal{R}_{\text{meta}}$) and transactional multi-file `SWE-EVO` AST verifier (Δ_{AST}).

- `meta/meta_architect.py`: Loop 2 Causal Program Slicer (δ_{slice}), Counterfactual Textual Backpropagation Engine, Surgical AST Patcher, and Library Crystallizer ($\mathcal{K}_{\text{lib}}$).

- `curriculum/challenger.py`: Loop 3 Stackelberg adversarial benchmark synthesizer utilizing Constructive Brute-Force Oracle Separation (O_{naive}) and Metamorphic Invariants.

- `distill/trajectory_exporter.py`, `grpo_loop4_trainer.py`, & `adapter_router.py`: Loop 4 JSONL trajectory exporter, GRPO LoRA trainer (`lora_loop4_adapters/system1_distilled`), dynamic domain-indexed router ($\Delta\theta_d$), and empirical telemetry logs (`track1_distillation_metrics.json`, `track2_mutation_metrics.json`, `macro_epoch2_closure_metrics.json`, `step2_crystallization_metrics.json`, `step3_metamorphic_metrics.json`, `step4_swe_evo_metrics.json`).

References

[1] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in AI safety," *arXiv preprint arXiv:1606.06565*, 2016.

[2] V. S. Borkar, "Stochastic approximation with two time scales," *Systems & Control Letters*, vol. 29, no. 5, pp. 291–294, 1997.

[3] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[4] M. Dennis *et al.*, "Emergent complexity and zero-shot transfer via unsupervised environment design," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 13049–13061, 2020.

[5] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems," *arXiv preprint arXiv:2408.08435*, 2024.

[6] D. Kahneman, *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

[7] O. Khattab *et al.*, "DSPy: Compiling declarative language model calls into self-improving pipelines," in *International Conference on Learning Representations (ICLR)*, 2024.

[8] D. R. MacIver, Z. Hatfield-Dodds, *et al.*, "Hypothesis: A new approach to property-based testing," *Journal of Open Source Software*, vol. 4, no. 43, p. 1891, 2019.

[9] J.-B. Mouret and J. Clune, "Illuminating search spaces by mapping elites," *arXiv preprint arXiv:1504.04909*, 2015.

[10] M. Robeyns, M. Szummer, and L. Aitchison, "A self-improving coding agent," *arXiv preprint arXiv:2504.15228*, 2025.

[11] Z. Shao *et al.*, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024.

[12] H. von Stackelberg, *Marktform und Gleichgewicht*. Julius Springer, 1934.

[13] X. Yin, X. Wang, *et al.*, "Gödel Agent: A self-referential agent framework for recursive self-improvement," *arXiv preprint arXiv:2410.04444*, 2024.

[14] M. Yuksekgonul *et al.*, "TextGrad: Automatic differentiation via text," *arXiv preprint arXiv:2406.07496*, 2024.

[15] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, "STaR: Bootstrapping reasoning with reasoning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 15476–15488, 2022.